Arya College of Engineering and IT, Jaipur

HackAgon: A Web-Based Platform for Hosting Skill-Based Live Coding Competitions with Escrowed Prize Pools and Proctored Matches

Hiten Sharma
Roll No. 24EARCS062
Dept. of CSE
Jaipur, India

Harsh Sharma
Roll No. 24EARCS055
Dept. of CSE
Jaipur, India

Akshat Rathore
Roll No. 24EARCS014
Dept. of CSE
Jaipur, India

Hemant Singh Tanwar
Roll No. 24EARCS058
Dept. of CSE
Jaipur, India

Arjun Suman
Roll No. 24EARCS028
Dept. of CSE
Jaipur, India

Under the Guidance of
Er. Ram Babu Buri, Dept. of CSE

***Abstract*—Online coding contests are almost always run by the platform that owns them. Students, college clubs and small organizations cannot easily host their own contests, set their own rules and prize splits, and be sure that both the money and the fairness of the event are handled correctly. This paper presents HackAgon, a web application in which any registered user can create a live coding room or tournament, and in which the platform, not the host, controls the money and the integrity of the contest. The system provides free practice rooms, one-versus-one duels, team formats, and paid tournaments whose entry fees and prize pools are held in an append-only ledger and escrow until results are locked, after which winners are paid automatically through Razorpay. An optional proctored mode combines webcam snapshots, tab and fullscreen monitoring, and a lock state stored on the server so that a violation cannot be undone by refreshing the browser. Submissions are executed in an isolated judge service, scores and ranks are computed with integer arithmetic and a fixed tie-break rule, and a Learning module records every failed question for later review. We describe the requirements, architecture, data model, scoring and payout algorithms, anti-cheating design, security and privacy measures, and a twelve-week team development plan. We also define an evaluation methodology with measurable targets. The paper is a design and engineering report; it does not claim measured results that have not yet been collected.**

***Keywords*—online judge, programming contest, escrow, online proctoring, Razorpay, skill-based competition, web application, ledger, anti-cheating**

I. INTRODUCTION

Competitive programming has become a normal part of how students learn to code and how companies screen candidates. Large platforms offer practice problems, ranked contests and learning material, and they have built large communities around them. However, in almost all of these systems the contest belongs to the platform. The platform chooses the problems, the schedule, the rules and the rewards. A student club that wants to run an inter-college contest, a teacher who wants a quick class test, or a small company that wants to sponsor a hiring challenge must either adapt to the fixed rules of a large platform or build a judge, a scoreboard and a payment flow from scratch.

Building such a system is harder than it first appears. A contest needs a safe way to run untrusted code, a trustworthy clock, a scoreboard that stays correct under load, and rules that every participant can see before joining. When money is involved, the problem changes in kind. Entry fees must be collected, prize money must be guaranteed, results must be agreed, and winners must be paid correctly even when a payment fails or a webhook arrives twice. When fairness is involved, the host needs tools to detect and remove cheaters, and the participants need protection from a host who removes a leading player without proof.

HackAgon is our answer to this gap. The guiding idea, stated in the product requirements, is a single place to practice, compete and learn, where the host controls the contest and the platform controls the money and the integrity. A host decides the questions, the languages, the duration, the format and the prize split. The platform holds the money in escrow, runs the judge, enforces proctoring rules on the server, and settles payments according to the rules that were published before registration opened.

A. Problem Statement

Existing platforms do not let ordinary users host their own contests with their own rules and prize splits while also guaranteeing that money and fairness are handled for them. The problem addressed in this paper is the design of a web platform that (a) lets any user create a contest in under three minutes with sensible defaults, (b) holds all entry fees and prize money in platform escrow until the contest ends, (c) makes casual cheating impractical through proctoring, and (d) keeps users returning through duels, a learning library, a mistakes review page and achievements.

B. Contributions

This paper makes the following contributions.

- A requirements model for a host-driven contest platform with five user roles, seven contest formats and options, and a set of measurable success metrics (Sections III and XV).
- A layered architecture that separates the browser, the HTTP API, the service layer, the data store and four external services, together with a data model of more than twenty tables (Sections IV and V).
- A server-authoritative match engine in which time, scoring and lock state are decided on the server, so that client-side manipulation gains nothing (Sections VII and X).
- A money design based on integer paise, basis-point prize splits, an append-only ledger, idempotent webhooks and a deterministic rounding rule, with the invariants that make payouts auditable (Section IX).
- A proctoring design that combines webcam snapshots, browser event detection, a network grace period and similarity detection, with an honest statement of its limits (Section X).
- A skill-only rule set that keeps paid contests free of any element of luck, and a development plan for a five-member team over twelve weeks (Sections XIV and XVI).

C. Organization of the Paper

Section II reviews related work. Section III states the requirements. Sections IV to VI describe the architecture, the data model and the account system. Sections VII and VIII describe the match engine and the judge. Section IX covers tournaments, escrow and payouts. Section X covers proctoring. Sections XI to XIII cover learning, security and the user interface. Sections XIV to XVI describe the team plan, the evaluation methodology and the legal and ethical considerations. Section XVII lists limitations and future work, and Section XVIII concludes.

II. RELATED WORK

The design of HackAgon draws on four areas of existing practice: online judges and contest systems, online proctoring, electronic payment and escrow design, and the educational use of programming practice. We describe each area in general terms and explain how our design relates to it. We do not compare against the commercial features of specific products, because those features change often and cannot be verified from the project documents.

A. Online Judges and Contest Systems

An online judge accepts source code, compiles or interprets it in a restricted environment, runs it on hidden test cases and returns a verdict such as accepted, wrong answer, time limit exceeded or runtime error. Contest systems add a clock, a scoreboard and a scoring rule on top of the judge. Common scoring rules include penalty-based ranking, in which solved count is the primary key and total time with penalties is the tie-break, and points-based ranking, in which each problem carries a value that may decay with time. HackAgon uses points by difficulty, a time bonus and a fixed tie-break on the earlier last correct submission, because this rule is easy to explain on the rules page and easy to verify afterwards.

Open-source judge services allow a project to avoid building its own sandbox. The Judge0 project, for example, exposes a REST interface that accepts code, language and input and returns status, output, time and memory, while enforcing CPU, memory and time limits and blocking network access [1]. Building an untrusted-code sandbox from scratch is slow and easy to get wrong, so our design treats the judge as an external service behind a small client module and keeps the option to move from a hosted instance to a self-hosted one.

B. Online Proctoring

Proctoring systems try to preserve the integrity of remote assessments. Typical techniques include camera monitoring, screen or tab monitoring, identity checks and post-hoc review by a human. Browser platforms provide the building blocks: the Page Visibility API reports when a page becomes hidden [2], and the media devices interface lets a page request camera access and capture frames [3]. These signals can be bypassed by a determined cheater, for example by using a second device or a virtual machine, so a realistic design must combine several weak signals with human review and must state its limits clearly. HackAgon follows this layered approach and keeps the host's right to eliminate a player as the final control.

Source-code plagiarism detection is a second integrity layer. Document fingerprinting by winnowing selects a representative subset of hashed k-grams so that near-copies share fingerprints even after small edits [4]. Our similarity check between submissions in the same room is intended to follow this family of methods and to produce a flag list for host review rather than an automatic penalty.

C. Payments, Escrow and Ledgers

Systems that collect money from many people and pay a few of them are marketplaces in the payments sense. They need a record of every money movement that cannot be silently changed. Double-entry bookkeeping has long solved this problem in accounting: every movement is recorded as an immutable entry, and balances are derived from entries rather than stored and edited. We adopt the same principle through an append-only wallet ledger. Payment gateways deliver outcomes asynchronously through signed webhooks, and the same event can arrive more than once, so handlers must be idempotent. Razorpay provides orders, signed webhooks, payouts and a fund account validation feature for checking that a UPI identifier or bank account exists [5]. Our design relies on these features and on database transactions to keep the ledger consistent.

D. Authentication and Credential Storage

Passwords must be stored as salted, adaptive hashes. The bcrypt scheme was designed for this purpose and allows the work factor to rise as hardware improves [6]. One-time passwords are widely used for second-factor and ownership checks; the HOTP specification describes how short numeric codes can be generated and validated [7]. Our system stores only a hash of each one-time code, limits attempts, and enforces an expiry, which are standard defensive measures described in web security guidance [8].

E. Gap Addressed by This Work

The common thread in the above areas is that each solves one part of the problem. Judges solve execution, proctoring tools solve monitoring, and payment gateways solve collection and payout. What is missing for student clubs and small hosts is a single product that combines them under a host-controlled, skill-only rule set, with the platform acting as the neutral holder of money and evidence. HackAgon is designed to fill that gap.

III. REQUIREMENTS ANALYSIS

A. Target Users and Roles

The product serves five groups of users. Students and learners want practice, duels, a review of past mistakes and free learning books. Competitive players want rated matches, tournaments with prizes and achievements. Individual hosts want to run a quick contest for friends or a class. Organizations and college clubs want to run a large, paid, proctored tournament with rules and payouts. Platform administrators moderate users, resolve payment disputes and manage the question bank. These needs map to four roles, shown in Table I. A user holds the Player role by default and becomes a Host the first time a room is created; organization status is requested through a form and approved by an administrator.

TABLE I
USER ROLES AND CAPABILITIES

Role	Capabilities
Player	Join rooms, duels and tournaments; practice; use the learning library; earn achievements
Host	Any player who creates a room or tournament; manages players; eliminates cheaters
Organization host	Host account with an organization name and extra verification; larger tournaments
Admin	Manages question bank, books, users, disputes, payouts and flagged cheating

B. Functional Requirements

The functional requirements are grouped by subsystem. Each requirement in the product document has an acceptance criterion that can be tested in the browser, and the most important ones are summarized below.

- **Accounts.** Signup with name, username, email, mobile number, date of birth and password; verification by a six-digit code sent to both email and mobile; login by email or username; forgot-password by one-time code; profile editing with fresh verification when email or mobile changes.
- **Rooms.** A room creation form in which every field has an editable default; a side panel that lists saved questions filtered by language, topic and difficulty; public rooms listed in an Arena and private rooms reachable by invite code.
- **Formats.** Solo, Duo, Squad (three to five members) and Duel (one versus one); team score counts each question once per team.
- **Tournaments.** Entry fee and prize pool in rupees, prize split presets or a custom split totalling 100 percent, escrow, refunds, payout, and a rules page that must be accepted before payment.
- **Tournament mode.** Webcam proctoring and tab lock, violation logging, a host dashboard and elimination with a stored reason.
- **Learning.** A My Mistakes page fed automatically by failed submissions, a library of legally free books, and achievements driven by thresholds stored in a table.
- **Administration.** Question and book management, user moderation, dispute review, payout failures and flagged cheating.

C. Non-Functional Requirements

The non-functional requirements are summarized in Table II. They include a scoreboard refresh under five seconds, a judge result under ten seconds for typical submissions, and capacity for five hundred concurrent players per server. Match state is stored in the database so that a restart does not lose a running contest. Money is stored as integer paise, never as floating point.

TABLE II
NON-FUNCTIONAL REQUIREMENTS

Area	Requirement
Security	Hashed passwords, parameterized queries, CSRF protection, HTTPS only, encrypted payout details, rate limits on OTP and login
Payments	Webhooks verified by signature, idempotent handling, full audit ledger
Performance	Scoreboard refresh under 5 s; judge result under 10 s for typical submissions; 500 concurrent players per server
Reliability	Match state kept in the database so that a restart does not lose a running contest
Privacy	Consent for webcam and data use; retention limits; account deletion on request
Compatibility	Latest Chrome, Edge and Firefox on desktop; tablet and phone for browsing only

D. Success Metrics

Four metrics define success. At least 40 percent of new accounts should join a first room within seven days of signup. All tournaments that complete must have all payouts settled correctly, a target of 100 percent. Fewer than 5 percent of

proctored tournaments should have a confirmed cheating incident. Each active host should create at least two rooms per month. A design rule supports honest measurement: every counter in the product, such as rooms won, rank and achievements, starts at zero for every new account and grows only from real activity, and no placeholder number is shown anywhere.

E. Constraints and Assumptions

Some of the requested features cannot be built with plain HTML pages alone, and the design assumes small additions that are unavoidable. Tab-change detection needs browser events and therefore a small JavaScript file. Webcam proctoring needs the browser camera interface and periodic snapshots posted to the server. A live scoreboard and countdown need polling or server push. Running player code safely needs a sandboxed judge. A plain text area is a poor code editor, so an optional editor library is allowed. Languages are limited in the first version to Java, Python, C++, C and JavaScript. Native mobile applications, team chat, video calls between players and live spectating are explicitly out of scope.

IV. SYSTEM ARCHITECTURE

A. Architectural Overview

HackAgon is organized in layers, shown in Fig. 1. The browser layer contains the HTML pages, stylesheets and small vanilla JavaScript files for timers, polling, the editor and the proctoring script. The API layer exposes HTTP routes grouped by feature in a REST-style interface [9]: authentication, rooms, play, money, proctoring, administration, content and AI hints. The service layer holds the business rules: one-time codes, scoring, ledger operations, achievements and the judge client. The data layer is a relational database accessed only through a repository module that uses parameterized statements. Four kinds of external service sit outside the application: a code judge, a payment provider, message delivery for email and SMS, and object storage for webcam snapshots.

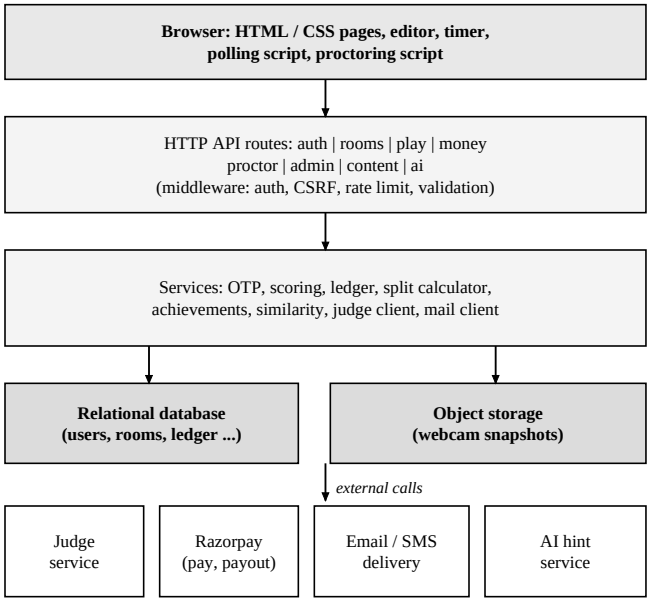


Fig. 1. Layered architecture of HackAgon.

B. Technology Choices

The first version of the product document specified Java servlets and JSP on Apache Tomcat with a JDBC database. During the planning audit the team found that the existing code base was a Node.js and Express application with plain HTML pages and a MySQL database. The team plan therefore recommends keeping the existing stack, because it already exists and deploys easily, and updating the requirements to match. This paper describes the architecture in a way that holds for both choices, and Table III records the current stack and the original Java design side by side. The choice is a decision about implementation effort and does not change the behaviour of the system. The requirements, plan and audit are recorded in the team's project documents [10]–[12]. The deployment uses managed services: the Render hosting platform for the web service [16], TiDB Cloud as a MySQL-compatible managed database [14], Brevo as the transactional email provider that replaces Gmail SMTP before launch [15], and the Groq API for AI hints [13].

TABLE III
CURRENT STACK AND ORIGINAL JAVA DESIGN

Layer	Current code base	Original design
Server	Node.js, Express	Jakarta Servlets, JSP, Tomcat
Database	TiDB Cloud (MySQL-compatible) via mysql2	PostgreSQL via JDBC, HikariCP
Password hash	bcryptjs	BCrypt library
Session	JSON Web Token, secure cookie	Server session, HttpOnly cookie
Email / SMS	Nodemailer with Brevo; MSG91	JavaMail with Gmail SMTP; MSG91
AI hints	Groq API	Not in original design
Judge	Hosted judge service, own VM later	Judge0 CE, hosted then own VM
Payments	Razorpay	Razorpay Java SDK

Layer	Current code base	Original design
Hosting	Render	Render Docker service

C. Project Structure

The server entry file wires configuration, the database pool and the routes. The library folder holds one module per concern: authentication, one-time codes, mail, the domain rules for rooms and scoring, the judge client, the repository and utilities. Routes are thin: they read and validate input, call the library, and return JSON. This separation follows the working rule from the implementation plan that handlers read input and call a service, services call data access code, and pages only display data. A rule that every secret, such as the database URL, payment keys, SMS key and mail password, is read from environment variables and never from the repository protects against accidental disclosure.

D. Live Updates by Polling

The scoreboard, the room list and the lock state use polling every three to five seconds and return small JSON documents. Polling works on a typical hosting platform without extra setup, is easy to cache for short intervals, and degrades gracefully when a connection is slow. If polling load later becomes a problem, Server-Sent Events can replace it with the same payloads. At 500 concurrent players polling every four seconds, a server receives about 125 requests per second for the scoreboard alone, which motivates the short cache described in Section XV.

V. DATA MODEL

The data model has more than twenty tables. All identifiers are auto-incrementing integers, timestamps are stored in UTC, and money is stored as integer paise. Every table has a creation time. Foreign keys and indexes exist on every join column, and composite indexes on (room, user) and (room, creation time) support the scoreboard queries. Table IV lists the main groups.

TABLE IV
MAIN TABLES AND THEIR PURPOSE

Group	Tables	Notes
Identity	users, otp_codes, payout_accounts	Unique email, username and mobile; hashed codes; encrypted payout data
Content	questions, test_cases, books	Owner is null for platform questions; hidden cases are marked
Contest	rooms, room_settings, room_players, teams, room_questions	Status: draft, open, live, finished, cancelled
Judging	submissions	Stores code, verdict, passed cases, time
Integrity	violations, snapshots	Snapshot image is in object storage; table keeps the key
Money	tournaments, prize_splits, wallet_ledger, payments, payouts	Ledger is append only; splits in basis points
Engagement	mistakes, achievements, user_achievements, notifications	Thresholds editable in the table

A. Rooms and Players

A room has a host, a name, a format, a player limit, a team size, a duration, an optional per-question limit, a visibility, an invite code, a tournament-mode flag, a status and a start time. Languages, topics and questions are stored in join tables with positions so that the host's ordering is preserved. The room_players table records, for each participant, a state and a score. The state is one of joined, playing, locked, eliminated or finished, and a locked reason is stored with it. Because the lock lives in this table, it is a server fact and not a browser fact.

B. Tournaments and Splits

A tournament is a room with money attached. Its row stores the entry fee, the prize pool, the platform fee, the minimum number of players, the time at which the pool was funded and the time at which results were locked. Prize splits are stored per rank in basis points, where 10000 basis points equal 100 percent, and the sum must equal 10000. Basis points avoid fractions and let the split calculator use integer arithmetic only.

C. Reference Data and Counters

The database ships with reference data only: topics, achievements, books and a starter set of real questions per topic, so that a new host has something to choose on day one. No fake users, scores or room counts are inserted. Profile counters such as rooms won and tournaments won are calculated from room_players and tournaments, which guarantees that they start at zero for every new account and cannot drift from the underlying records.

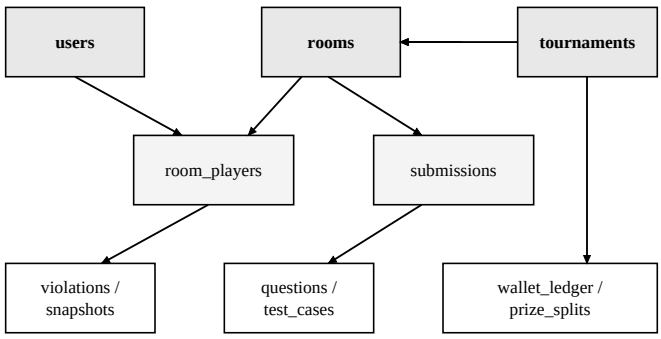


Fig. 2. Simplified entity relationships (arrows point from the owning table to the dependent table).

VI. ACCOUNTS, AUTHENTICATION AND PROFILE

A. Signup and Verification

A visitor supplies name, username, email, mobile number, date of birth and password. The server generates a six-digit one-time code and sends it to both the email address and the

mobile number. The account is created in an unverified state and becomes active only after both channels succeed. The code expires after five minutes, allows five attempts, and can be resent after sixty seconds. These limits are enforced on the server, which stores only a hash of the code together with the number of attempts and a used flag. A wrong or expired code never activates an account, and a user cannot log in until verification is complete.

TABLE V
ONE-TIME CODE POLICY

Parameter	Value	Reason
Length	6 digits	Easy to type on a phone
Lifetime	5 minutes	Limits the window for interception
Attempts	5	Limits guessing; 10^6 codes
Resend delay	60 seconds	Limits message cost and abuse
Storage	Hash only	A database leak does not reveal live codes
Channels	Email and SMS	Both must verify at signup

With a six-digit code and five attempts, the probability that a random guess succeeds within one code lifetime is at most 5 in 10^6, or 0.0005 percent. Rate limits on the verification route add a second barrier against distributed guessing across many codes.

B. Login and Sessions

Login accepts an email address or a username with a password. Passwords are stored as salted bcrypt hashes and are never stored or logged in plain text. Sessions use a signed token carried in a secure, HttpOnly cookie so that scripts in the page cannot read it. Forgot-password uses a one-time code sent to the registered email. Cross-site request forgery is addressed with tokens on state-changing requests, and the rate limiter slows repeated login failures. A planned security review tests injection, session fixation and token misuse before launch.

C. Profile and Payout Details

The editable profile fields are display name, email, mobile number, bio and preferred languages. Changing email or mobile requires a fresh code. The profile shows rooms played and won, tournaments won, rating and achievements, all computed from real records. Payout details, a UPI identifier or a bank account with IFSC code, are added on the profile and verified as described in Section IX. They are stored encrypted, and changing them requires a fresh code and puts payouts on hold for twenty-four hours, which limits the damage from a stolen session.

D. Age Rule

Date of birth is captured at signup. Only users aged eighteen or older may enter or host paid tournaments. The check is made by the server at registration and at tournament creation, not only in the browser.

VII. ROOMS AND THE MATCH ENGINE

A. Room Creation

A room is a single live contest and is the base unit of the system. The creation form gives every field an editable default so that a host can publish quickly, which supports the goal of creating a room in under three minutes. Table VI lists the fields. A room cannot be published without a name of three to fifty characters, at least one language, at least one topic or question, and a valid duration. After the contest starts, the host cannot change settings, except to eliminate a player. The rules, entry fee and prize split are visible on the room page before a player joins.

TABLE VI
ROOM CREATION FIELDS AND DEFAULTS

Field	Options	Default
Room name	Free text, 3 to 50 chars	Empty (required)
Format	Solo, Duo, Squad, Duel	Solo
Players / teams	2 to 100 solo; squad size 3 to 5	10
Languages	Java, Python, C++, C, JavaScript	Java
Topics	Arrays, strings, linked lists, stacks, queues, trees, graphs, sorting, recursion, dynamic programming	Arrays
Questions	Side panel or auto-pick	Auto-pick 5
Total duration	15 minutes to 6 hours	60 minutes
Time per question	None, or 5 to 120 minutes	None
Visibility	Public, or private with invite code	Public
Prize pool / entry fee	Amount in INR, or none	0 / none
Tournament mode	Off, or on (webcam + tab lock)	Off

B. Question Side Panel

While creating a room or tournament, the host opens a side window that lists saved questions filtered by language, topic and difficulty. The host ticks the questions to include, previews each one and reorders them. The panel reads from the question bank described in Section XI. Hosts can also contribute private questions that only they can use in their own rooms.

C. Room Lifecycle

A room moves through a small set of states, shown in Fig. 3. A background timer on the server changes the status from open to live at the start time and from live to finished at the end of the duration. Because the state is stored in the database, a server restart resumes the timer from stored values instead of losing the contest. A tournament that is cancelled before it starts, or that does not reach its minimum number of players, moves to the cancelled state and triggers refunds.

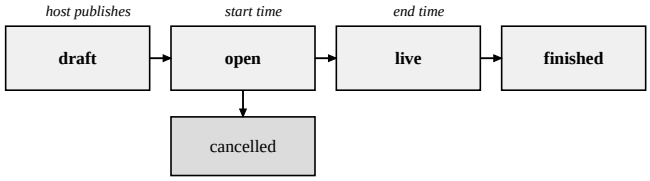


Fig. 3. Room lifecycle states and transitions.

D. Match Flow

The host publishes the room and it appears in the Arena list or becomes reachable by invite code. Players join until the start time or until the room is full. At the start, all players see the first question and a code editor in their chosen language. A submission runs in the judge against hidden test cases, and the player sees pass or fail per case. Scoring is described below. A live scoreboard updates every few seconds, and at the end final ranks are fixed and stored.

E. Formats

- **Solo.** Individual ranking.
- **Duo and Squad.** The team score is the sum of the team's solved questions, and a question counts once per team even if several members solve it.
- **Duel.** Exactly two players race on the same question set. The first to pass all cases, or the player with the higher score at the time limit, wins. Duels are created by invite or matched automatically against another waiting player.

Platform-run daily rooms and always-open practice duels use the question bank, are free and have no prize. A practice mode serves questions by topic and difficulty with no timer or an optional timer. Wrong answers are saved automatically for the Learning page.

F. Scoring and Tie-Break

Points are awarded per question by difficulty, with a time bonus for early solutions. Let d be the difficulty class of a question, p_d its base points, T the total duration, t the time from the start of the match to the first accepted submission, and b the maximum bonus. A question solved by a player or team contributes

$$s = p_d + \blacksquare\, b \cdot (1 - t / T) \blacksquare \qquad (1)$$

and the room score is the sum of s over solved questions. The values of p_d and b are configuration parameters and are not fixed by this paper. Ties on total score are broken by the earlier time of the last correct submission. Because both inputs are decided by the server, the rank order is reproducible from the stored submissions. A question already solved is never scored twice, and the scoring update for a submission runs inside one database transaction.

G. Server-Authoritative Time

The countdown shown in the browser is only a display. The server decides whether a submission is on time using the room's start time and duration. The page asks the server for the remaining time on load and on every poll, so changing the computer clock gains nothing. Submissions that arrive after the end time are rejected even if the browser still shows time left. This rule is also what makes the tie-break fair, since all timestamps come from one clock.

VIII. CODE JUDGING SUBSYSTEM

Executing untrusted code on the application server is the highest technical risk in the project. The design therefore delegates execution to a sandboxed judge service and keeps the application server out of the execution path. The judge client module sends the code, language and test input and receives status, output, time and memory.

A. Submission Pipeline

Fig. 4 shows the pipeline. The browser posts code, language and question identifier. The server checks that the user is in the room, that the room is live, and that the player is neither locked nor eliminated. It saves the submission as pending, runs it against each test case with the time and memory limits of the question, compares the output with the expected output, sets the verdict and the number of passed cases, and recalculates the score in one transaction. The browser then polls for the verdict and the scoreboard.

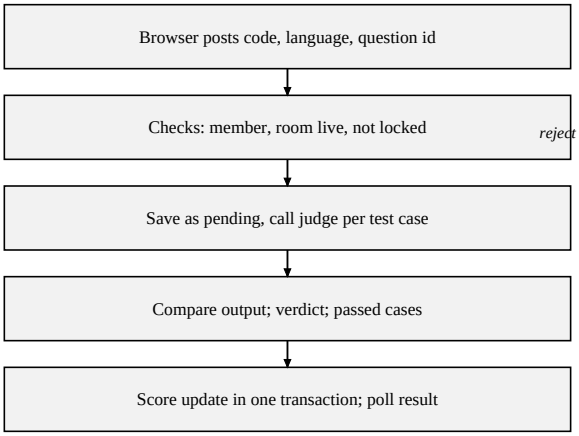


Fig. 4. Submission and judging pipeline.

B. Verdict Mapping

The judge returns a status code per test case. The service maps these to a small set of verdicts that players understand: accepted, wrong answer, time limit exceeded, memory limit exceeded, runtime error and compilation error. The verdict for a submission is accepted only if every case passes. Hidden cases are never revealed, but the player sees pass or fail for each case, which supports debugging without leaking the expected output.

C. Concurrency and Abuse Control

A contest produces bursts of submissions near the end of the match. The service uses a small worker pool so that judge calls do not block web requests, limits the number of concurrent runs, and allows only one pending submission per player per question to stop spamming. If the hosted judge caps daily runs or speed, the system queues submissions and caps concurrency. The product document sets the trigger for moving to a self-hosted judge: before the first paid tournament,

or as soon as the hosted plan's daily limit becomes a problem, whichever comes first.

D. Sandbox Properties Required

The judge must run with no network access, with limits on CPU time, memory and wall-clock time, and with no shared filesystem between runs. These properties are required because contestants can submit code that attempts to read files, open connections, fork processes or run forever. The test plan includes a set of known good and bad solutions per language that must always receive the expected verdict, as well as infinite loops and very large inputs.

E. Stored Code and Reuse

Every submission is stored with its code. The stored code serves three purposes: it lets the player see the last attempt on the My Mistakes page, it supports similarity checks between players in the same room, and it gives administrators evidence in disputes.

IX. TOURNAMENTS, ESCROW AND PAYOUTS

Any user can host a tournament, with or without an entry fee. Organizations use the same flow through a Host Tournament option that adds organization details and larger limits. The central design principle is that the platform, not the host, holds the money, and that every movement of money is recorded in a ledger that is never edited.

A. Money Flow

The flow has six steps, shown in Fig. 5. First, the host funds the prize pool by paying the full amount to the platform through Razorpay, and an unfunded tournament remains a draft. Second, players pay entry fees at registration, and the platform holds them. Third, the tournament runs and no money moves. Fourth, results are locked after the host confirms them or after an automatic confirmation window of twenty-four hours passes with no dispute. Fifth, the platform pays each winner according to the published split. Sixth, if the tournament is cancelled before it starts or does not reach its minimum players, entry fees and the prize pool are refunded in full. A player eliminated for cheating receives no refund.

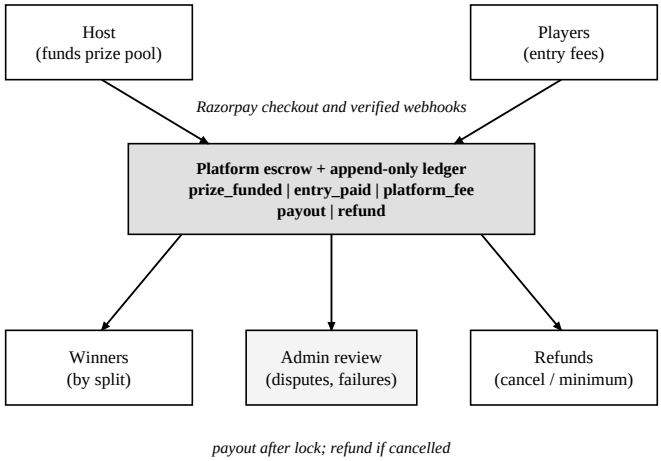


Fig. 5. Money flow through platform escrow.

B. Platform Fee and Prize Distribution

The platform fee is 1 percent of the total prize pool, deducted from the pool before winners are paid. Prize splits apply to the pool after the fee. Entry fees are not charged a fee, and all amounts are shown to the host and the players before payment. The host picks one preset or builds a custom split, and all splits must total 100 percent. The split is saved with the tournament and cannot be changed once registration opens. Table VII lists the presets.

TABLE VII
PRIZE SPLIT PRESETS

Preset	1st	2nd	3rd	Others
Winner takes all	100%	0%	0%	0%
Top 3	50%	30%	20%	0%
Top 5	40%	25%	15%	10% each, 4th and 5th
Custom	Host sets percent per rank			

C. Integer Payout Algorithm

Let P be the prize pool in paise, f the fee, N the net pool, and w_i the split of rank i in basis points with the sum of all w_i equal to 10000. The platform computes

$$f = \blacksquare\, P / 100 \,\blacksquare, \quad N = P - f \qquad (2)$$

$$a_i = \blacksquare\, N \cdot w_i / 10000 \,\blacksquare, \quad r = N - \Sigma\, a_i \qquad (3)$$

The remainder r, at most a few paise, is added to the first rank so that no paise are lost or created. The payout to rank 1 is then $a_1 + r$. All arithmetic is integer arithmetic in one service class, which removes the rounding errors of floating point money.

Worked example 1. A prize pool of Rs. 5,000 is 500,000 paise. The fee is 5,000 paise, so N = 495,000. With the Top 3 split the payouts are 247,500, 148,500 and 99,000 paise, which sum to 495,000 with no remainder.

Worked example 2. A pool of 100,001 paise gives f = 1,000 and N = 99,001. The Top 3 amounts are 49,500, 29,700

and 19,800, which sum to 99,000. The remainder of 1 paise goes to rank 1, which therefore receives 49,501 paise. The total paid out equals *N* exactly.

D. Ledger Invariants

The wallet ledger is append only; entries are never updated or deleted. Each entry has a kind: prize funded, entry paid, platform fee, payout or refund. Each money event is an insert inside a database transaction. For a tournament, credits minus debits must equal the amount still held, and when the tournament is settled the balance returns to zero. Let *C* be the sum of prize_funded and entry_paid entries, and *D* the sum of platform_fee, payout and refund entries. Then

$$held = C - D \geq 0, \quad held = 0 \text{ at settlement} \qquad (4)$$

A nightly check job compares the ledger with the payment provider's records and reports any difference to the administrators. The acceptance criteria require that the sum of all payouts never exceeds the money held for the tournament, that a payout failure is retried and then shown to an administrator, and that a player's winnings are never lost.

E. Idempotent Webhooks

The payment provider reports the outcome of payments through webhooks. The handler first verifies the request signature, then looks up the provider's event or payment identifier. If the identifier has already been processed, the handler returns success without changing anything. Otherwise it applies the change and records the identifier in the same transaction. This makes duplicate delivery safe, which is required because delivery is at least once. A paid tournament cannot open registration until the full prize pool payment is confirmed by a verified webhook, and a player joins a room only after the entry payment is confirmed.

F. Payout Details and Verification

Every player entering a paid tournament, and every winner, needs verified payout details. The player enters a UPI identifier or a bank account number with IFSC. A one-time code is sent to the mobile number registered on the player's profile. The platform then calls the provider's fund account validation, a small test transfer, to confirm that the account exists and to obtain the registered account holder name. The holder name is compared with the profile name, and a clear mismatch is flagged for administrator review before any payout. The registered mobile number behind a UPI identifier is not exposed by the payment network or by the provider, so the system cannot compare it directly. The one-time code to the profile number together with the name returned by validation is the reliable substitute, and the design states this limit openly.

G. Host Powers and Disputes

A host may eliminate any player at any time with a reason that is stored and shown to the player. Eliminated players cannot rejoin, and their score is removed from the ranking. To protect players from abuse, for example removing the leader without proof, a player can raise a dispute that blocks payout until an administrator reviews it. Disqualification for unfair means is final for payouts: no refund and no prize.

H. Rules Page

Before a player can pay or join, one page shows the entry fee, prize pool, prize split, languages, topic count, duration, whether tournament mode is on, elimination rules, refund rules and platform fee. Joining requires ticking an acceptance box, and the acceptance is stored with a timestamp. Players therefore always see the rules before paying, and the platform holds evidence of what was accepted.

I. Failure Handling

Payment systems fail in several ways: a payout can be rejected, a webhook can be delayed, a player's bank can be unavailable, and a server can restart between two steps. The design handles these by retrying payouts with a bounded number of attempts, then showing the failure on an administrator screen with a retry action, by keeping every step inside a transaction, and by treating the ledger, not the provider's dashboard, as the record of intent. A paid-mode feature flag in configuration allows the platform to switch off paid contests as a safety measure.

X. PROCTORING AND ANTI-CHEATING

Tournament mode is an option the host switches on when creating a room or tournament. When it is on, players must accept the conditions before joining, and the checks below run for the whole match. When it is off, none of these checks run.

A. Webcam Proctoring

Before the match, the player must grant camera access and pass a check that a face is visible. During the match, the browser sends a small snapshot to the server every twenty to thirty seconds, configurable by the host, and the video remains on screen in a small corner preview. If the camera is turned off, blocked or covered for more than fifteen seconds, the player is warned and then locked out. Snapshots are small JPEG images, about 320 pixels wide, to keep traffic low, and they are stored in object storage rather than on the web server's disk. They are visible only to the host and administrators and are deleted after a retention period of thirty days by a scheduled job.

B. Tab and Window Lock

The match page listens for tab switch, window blur, minimize, exit from fullscreen, and right-click, copy and paste. The match opens in fullscreen. On the first violation, the page shows a blocking screen and the player's match ends: the page shows a locked screen and the player cannot continue. The decisive design point is that the lock is stored on the server.

The browser sends the violation to a route that sets the player's state to locked, and the match routes refuse every request from a locked player. Refreshing the page, reopening it or logging in again does not bring the player back. Violations are logged with time and type and shown to the host.

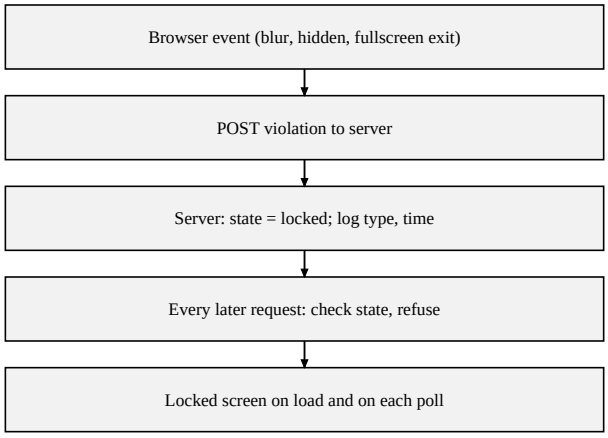


Fig. 6. Server-enforced lock sequence. The decision is stored on the server.

C. Network Grace Period

Honest players lose connections. A grace period of thirty seconds prevents a dropped network from being treated as cheating. The page sends a heartbeat, and a missing heartbeat for more than thirty seconds in a proctored room marks the player as disconnected and shows this to the host. The host can review the case and reinstate the player. The planned trials in weeks eight and ten measure false locks and tune the grace time.

D. Host Review and Platform Checks

The host dashboard shows a live list of players with camera status and violation count. The host can review a locked player's log and snapshots and either keep the lock or eliminate the player. Platform-run checks add similarity detection between submissions in the same room, flagged for host review, and a rule of one active session per account per match. Similarity flags are evidence for a human decision, not an automatic penalty, because legitimate solutions to short problems can resemble each other.

E. Threat Model

Table VIII lists the main cheating methods considered and the layered response. The design does not claim to stop all cheating. A second device or a second person off camera cannot be detected by these checks. The rules page says so, and the host's right to eliminate remains the final control. This honesty is itself a design decision: overstating a control would give hosts and players false confidence about money.

TABLE VIII
CHEATING METHODS AND RESPONSES

Method	Response	Residual risk
Switch tab or window	Visibility and blur events; server lock	Second device
Leave fullscreen	Fullscreen-exit event; server lock	Virtual machine
Copy and paste	Event blocking and logging	External notes
Cover or disable camera	Snapshots; 15-second rule	Camera loop or second person
Share solutions	Similarity detection; host review	Rewritten copies
Refresh to escape lock	Lock stored on server	None expected
Two sessions per account	One active session per match	Shared accounts
Change client clock	Server-side time	None expected

F. Privacy of Proctoring Data

Webcam images are personal data. The design requires an explicit consent screen, a retention limit of thirty days, access only for the host and administrators, and a privacy policy. Account deletion on request is a stated requirement. The data are used only for integrity review of the contest in which they were taken.

XI. QUESTION BANK, LEARNING AND ACHIEVEMENTS

A. Question Bank

Each question has a title, statement, topic tags, difficulty (easy, medium or hard), supported languages, a time limit, a memory limit, sample cases, hidden test cases and a reference solution. Topics cover arrays, strings, linked lists, stacks, queues, trees, graphs, sorting, recursion and dynamic programming. Administrators add and edit questions, and hosts may add private questions. The database ships with a seed set of real questions per topic. This seed is content for hosts to choose from and is not a fake user statistic. The plan grows the bank with medium and hard questions for every topic before launch.

B. My Mistakes

The Learning page has two parts. My Mistakes lists every question the user got wrong or failed to finish in past rooms, duels and practice. For each entry it shows the user's last submission, the failing case, the correct approach and a retry button. An entry is marked resolved when the user later solves the question. The list is empty for a new user, and the empty state is written with personality instead of a blank panel. Recording mistakes is a hook in the submission path: when a submission fails, the service inserts or updates a row in the mistakes table.

C. Library

The Library groups free books and study material by subject such as Java, Python, C++, data structures and algorithms. Each entry shows title, author, licence and a link or file. Only freely and legally available books, with an open licence or publisher approval, are listed, and administrators confirm the licence before adding one. The licence field is required, which turns a legal policy into a data constraint.

D. Achievements

Achievements unlock automatically from real activity, and every user starts with none unlocked and zero progress. Each achievement shows a progress bar toward the next milestone and a date earned, and appears on the public profile. Thresholds are stored in a database table so they can change without code changes. The achievement service runs after a room finishes or a submission passes, compares each metric with its threshold and inserts a row for the user when the threshold is met. Table IX lists the planned achievements.

TABLE IX
PLANNED ACHIEVEMENTS

Achievement	Condition
First Blood	Win a first room
Room Master	Win 20 rooms
Tournament Conqueror	Win a first tournament
Duelist	Win 10 duels
Perfectionist	Solve every question in a room
Clean Player	Finish 10 tournament-mode matches with no violation
Scholar	Retry and correctly solve 25 previously failed questions

Progress for a metric m toward a threshold k is displayed as $\min(m/k, 1)$ and is computed from live counts, so it cannot disagree with the records. The Room Master achievement, for instance, moves visibly after each win and unlocks at twenty.

E. AI Hints

An optional hint feature calls a hosted language model after a failed attempt to explain the likely mistake without giving a full solution. Daily limits per user keep usage within the provider's quotas. The hint is advice for learning; it never changes a verdict or a score, and the feature is not used inside proctored matches in a way that would give one player an advantage, a point that hosts should state in their rules.

XII. SECURITY AND PRIVACY DESIGN

A platform that holds other people's money and personal images is a valuable target. The security design combines standard web defenses with measures specific to contests. Table X lists the main threats and the planned controls, which follow the common categories in web application security guidance [8].

TABLE X
THREATS AND CONTROLS

Threat	Control
SQL injection	Parameterized statements everywhere; review of every route before launch
Cross-site scripting	Output escaping; no untrusted HTML in pages
Cross-site request forgery	Tokens on state-changing requests
Password theft	Salted bcrypt hashes; rate limits on login
OTP guessing	Hashed codes, expiry, five attempts, rate limit
Session theft or fixation	HttpOnly, secure cookie; new session at login; payout hold on detail change
Forged payment events	Signature verification; idempotency key
Payout detail leak	Encryption at rest; access limited to the payout job
Untrusted code	Isolated judge with resource limits and no network
Privilege abuse	Role checks on every admin and host route; admin scripts not reachable in production
Secret leakage	Environment variables; ignore files for secrets; no secrets in the repository

A. Operational Hygiene

The code audit performed in week one of the plan found several operational risks that are worth stating because they are common in student projects. A secrets file exists and must be confirmed as ignored by version control and never committed. Demo data in seed scripts and browser stores must not put fake users, scores or counters into the application. Scripts that create or repair administrator accounts must not be reachable in production. These items are tracked as explicit tasks rather than assumed to be fine.

B. Privacy and Data Retention

Personal data in the system include contact details, date of birth, payout identifiers and webcam images. The design limits collection to what the features need, encrypts payout details, restricts snapshot access to the host and administrators, deletes snapshots after thirty days, and supports account deletion on request. Separate staging and production environments ensure that payment and message tests never touch real users.

XIII. USER INTERFACE AND DESIGN SYSTEM

The product document gives design as much weight as function, because a competitive product needs a clear identity and because default generated interfaces look generic. The rules are that no page uses a purple-blue gradient, a generic glassmorphism card or a centred hero with three feature cards; fonts with personality are chosen instead of the most common defaults; and at least one section of each main page uses an asymmetric or unusual layout.

A. Direction and Tokens

The team evaluates several directions, including neo-brutalist, retro terminal, retro arcade, paper and ink, editorial, Swiss grid, soft claymorphism and a warm local style. One direction is chosen for launch after quick previews of the Arena page, and the chosen colours, fonts and spacing are recorded in a single style guide. All colours, fonts, spacing, radii, shadows and motion are defined as variables in one theme file, and no page may use a hard-coded colour or font. Changing the theme file changes the whole site, which also lets the team swap directions late in the project.

B. Navigation

Public pages are the landing page, login and signup with verification. After login, a bottom bar shows Tournaments, Arena, Duel, Learning and Profile. A Host button opens room

or tournament creation, and organizations see Host Tournament inside the Tournaments tab. The main pages are the Arena, creation forms, tournament details (the rules page), the match page, the duel page, the learning page, the profile, the host dashboard and the admin page.

C. Copy, Motion and Accessibility

Text is written as a friendly person from the user's city would say it: short sentences, specific, and free of marketing phrases. An empty Arena says there are no rooms yet and invites the user to start the first one. A failed code says it did not match and asks the user to try once more. Copy is reviewed as its own pass, separate from layout. Small details such as custom hover states, micro-animations, empty states and friendly errors are required, and each product has one signature moment, for example the countdown, the win screen or the lock screen. For accessibility, text contrast must be at least 4.5 to 1, focus states must be visible and pages must work at phone width. These are checked on the main pages before each phase closes.

D. Build Passes

Each page is built in four passes: layout, then type and colour, then motion and details, then copy. Separating the passes keeps reviews focused and prevents copy from being written to fit an unfinished layout. Design work is mapped to the delivery phases: direction and base layout in the first phase, room form, Arena and match page in the second, duel and learning screens in the third, consent and lock screens in the fourth, money screens in the fifth, and a final consistency pass in the sixth.

XIV. DEVELOPMENT METHODOLOGY AND TEAM PLAN

A. Phased Delivery

The project is delivered in six phases that each end with something that works end to end. Money is last on purpose because it carries the most legal and technical risk and should not block the rest of the product. Table XI lists the phases.

TABLE XI
DELIVERY PHASES

Phase	Contents
1	Signup with OTP, login, profile, question bank, practice mode
2	Room creation, solo matches, judge, scoreboard
3	Duo, squad, duels, Learning page, achievements
4	Tournament mode (webcam and tab lock), host dashboard, elimination
5	Escrow, entry fees, prize splits, UPI validation, payouts, admin tools
6	Organization hosting, hardening, load test, legal sign-off, launch

B. Team and Modules

The team has five members: Hiten Sharma, Harsh Sharma, Akshat Rathore, Hemant Singh Tanwar and Arjun Suman. The application is divided into ten modules, each with an effort weight of 2 or 3, and each member carries a total weight of 5 so that the load is equal. Table XII shows the module groups. The assignment of named members to module groups is a team decision and should be confirmed by the team; the paper uses the plan's member numbers.

TABLE XII
MODULE GROUPS PER MEMBER

Member	Modules	Weight
Member 1	Accounts and OTP; Rooms and matches	2 + 3
Member 2	Code judge and question bank; Admin and moderation	3 + 2
Member 3	Tournaments and money; Platform core and DevOps	3 + 2
Member 4	Proctoring and host dashboard; Duels and teams	3 + 2
Member 5	UI and design system; Learning, achievements and AI	3 + 2

Dependencies are scheduled early: rooms need the judge, tournaments need accounts for payout details, and every page needs the design system.

C. Working Practices

- **Branches.** The main branch is always deployable. Each task is a branch named module and task, merged by a pull request reviewed by another member.
- **Weekly rhythm.** A thirty-minute planning session on Monday, a ten-minute stand-up on Wednesday, a Friday demo of what works in the browser, then a short retrospective.
- **Contracts first.** Before two modules connect, the owners write the request and response in a shared contract file so that front end and back end work in parallel.
- **Definition of done.** The feature works in the browser, has a test or smoke check, shows no fake numbers, has hand-written empty and error messages, and is merged to main.
- **Shared files.** The schema, the server entry file and the shared browser scripts are changed only through the database owner and the design owner, to avoid merge conflicts.
- **Buffer.** About twenty percent of each week is kept free for bugs and review.

D. Twelve-Week Schedule

The plan assumes week one starts on 12 October 2026 and week twelve ends on 3 January 2027. Fig. 7 shows the three blocks and three milestones. Weeks one to four build the foundation and the first playable match; weeks five to eight add formats, proctoring and a paid tournament in test mode; weeks nine to twelve cover organizations, security, load testing, legal pages and launch.

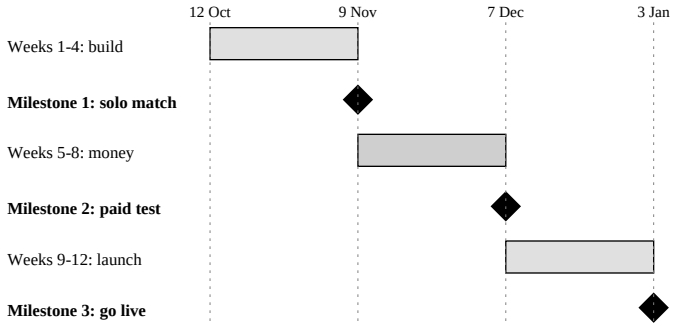


Fig. 7. Twelve-week schedule with three milestones (8 Nov, 6 Dec and 3 Jan).

E. Milestones

Milestone 1, on 8 November, requires two users to play a full solo room on staging, with a new account showing zero everywhere. Milestone 2, on 6 December, requires one full paid tournament in test mode: the host funds, players pay, cheaters are locked, results are confirmed, winners are paid by the split and the ledger balances. Milestone 3, on 3 January, requires production to be live and one real tournament to finish with correct payouts. Each milestone is a working demonstration on staging, not a list of finished tasks, and a missed milestone consumes the first week of the next block.

F. Risk Register

Table XIII summarizes the main project risks and fallbacks from the plan.

TABLE XIII
PROJECT RISKS AND FALLBACKS

Risk	Fallback
Stack differs between plans and code	Update requirements and plan in week 1
Judge service limits stall matches	Queue and cap concurrency; own VM using the trigger
Payment partner approval delays	Keep paid mode behind a switch; launch free tournaments first
False locks in tournament mode	30-second grace; host can reinstate; trials in weeks 8 and 10
Free hosting limits	Paid instance for production; snapshots in external storage
Uneven workload or exams	Equal weights; weekly buffer; reassign at planning
Merge conflicts on shared files	Only two owners edit shared files; others use pull requests

XV. EVALUATION METHODOLOGY AND TEST STRATEGY

The system is in development, so this paper does not report measured performance. Instead it defines how each requirement will be verified, so that the results can be reported honestly once collected. Evaluation has four parts: functional testing, performance testing, integrity testing and product metrics.

A. Functional and Unit Testing

Services that carry money or identity are covered by automated unit tests: scoring, ledger entries, prize split calculation and one-time codes. The split and ledger tests must cover rounding and failure cases, including pools that do not divide evenly, a single winner, a split that does not total 10000 basis points, and a transaction that fails halfway. Data access code is tested against a throwaway database. Pages and routes are tested by a manual checklist written for each phase, and an end-to-end script runs signup, room creation, join, solve and results. The judge is tested with a set of known good and bad solutions per language that must always receive the expected verdict.

B. Performance Testing

The performance targets are a scoreboard refresh under five seconds, a judge result under ten seconds for typical submissions, and five hundred concurrent players per server. The load test simulates 200 to 500 players in one room with polling, measures response time percentiles for the scoreboard route, and sends 200 queued submissions to the judge to tune concurrency. Indexes on (room, user) and (room, creation time) and a short cache for scoreboards are the planned optimizations. If the scoreboard is cached for c seconds and polled every p seconds, the worst-case staleness seen by a player is about $c + p$, so the sum must remain under five seconds.

$$c + p < 5\ s \qquad (5)$$

C. Integrity Testing

Proctoring is tested in the latest Chrome, Edge and Firefox with tab switch, minimize, fullscreen exit, camera denied and refresh after lock. The edge cases include re-login, two tabs, an unplugged camera and a slow network, and the pass condition is that locks survive while honest players are not punished. A ten-person trial match counts false locks, and the grace time is tuned from the result. Three mock matches with friends exercise the host dashboard end to end. For similarity detection, pairs of independently written and deliberately copied solutions measure how often copies are flagged and how often independent solutions are flagged by mistake.

D. Payment Testing

Payments are tested in the provider's test mode: failed payment, duplicate webhook, refund, payout failure and retry. The acceptance test for Phase 5 is a full cycle in which the host funds, three players pay, results lock, winners are paid per split and the ledger balances to zero for the tournament. Only after this passes does the team run one small real tournament with friends in live mode.

E. Metrics and Reporting

Table XIV lists the metrics that will be collected and their targets from the product document. Each metric is computed from stored records rather than from estimates.

TABLE XIV
EVALUATION METRICS AND TARGETS

Metric	Target	Source
Signup to first room joined within 7 days	At least 40%	users, room_players
Tournaments settled correctly	100%	ledger, payouts
Proctored tournaments with confirmed cheating	Under 5%	violations, host review
Rooms per active host per month	At least 2	rooms
Scoreboard refresh	Under 5 s	load test
Judge result, typical submission	Under 10 s	load test
Concurrent players per server	500	load test
False locks in trial match	To be measured; tune grace time	violations

Results will be reported with the number of runs, the environment and the date, and any target that is missed will be stated as missed. This avoids the common weakness of student project reports, which present planned numbers as achieved ones.

XVI. LEGAL, ETHICAL AND COMPLIANCE CONSIDERATIONS

A. Skill-Only Contests

Paid contests are acceptable in the design only while they remain skill-based. Any element of luck in scoring or prize selection would put the paid model at risk. The product document records that the team checked and found no restriction on paid coding contests when they involve no element of luck, and it advises keeping the source or written confirmation relied on, because the platform will hold other people's money. We note that interpretations of online gaming law can differ and change, so this paper does not give legal advice and the team should obtain written confirmation from a qualified lawyer before launching paid mode.

The design rules that keep a contest skill-only are the following.

- Every player in a room gets the same questions, and no question, prize or rank is decided by a random draw.
- No lucky draws, spin wheels, raffles or random bonus prizes.
- Ranks come only from correct solutions, time and the fixed tie-break rule.
- Random matching is allowed only to pick an opponent in practice duels, never in paid tournaments with a prize.
- The prize split is fixed and published before registration.

B. Holding Player Money

Holding user money in escrow may require a payment-aggregator arrangement or a nodal-style account under central bank rules. The plan therefore proposes using the provider's marketplace features, such as Route or its payout product, and confirming the structure with the provider and a lawyer. Paid mode sits behind a feature flag, and the fallback is to launch free tournaments first if approval is delayed.

C. Age, Consent and Data

Only users aged eighteen or older may enter or host paid tournaments, enforced from the stored date of birth. Webcam use requires explicit consent. Terms of service, a privacy policy, a refund policy and webcam consent text are written before launch, and a signup acceptance box records agreement. Learning material is limited to books with an open licence or publisher-approved free access, confirmed by an administrator.

D. Fairness to Participants

Hosts hold strong powers, so the design adds counterweights. Elimination needs a stored reason that is shown to the player. A dispute blocks payout until an administrator reviews it. Rules, fees and splits are visible before payment and cannot change after registration opens. The rules page states what proctoring cannot detect. These measures protect participants from both cheaters and careless hosts.

XVII. LIMITATIONS AND FUTURE WORK

A. Limitations

- **Proctoring is not proof.** A second device, a virtual machine or a second person off camera can defeat the checks. The system reduces casual cheating and supports human review, but it cannot guarantee a clean contest.
- **No measured results yet.** The system is described as designed and planned. Performance, false-lock rates and product metrics are targets until the evaluation in Section XV is run.
- **Judge dependence.** A hosted judge has daily limits and speed caps. Moving to a self-hosted judge needs a server that allows the required container privileges.
- **Legal uncertainty.** The treatment of paid contests and of holding user funds depends on law and on the payment partner's approval, which can change.
- **Scope.** The first version supports five languages, desktop browsers for play, and no native mobile applications, team chat or live spectating.
- **Polling cost.** Polling is simple but costs requests; at high concurrency, Server-Sent Events may be needed.

B. Future Work

- Rating system for rated matches, with a published update rule.
- Self-hosted judge on a dedicated virtual machine, and support for more languages.
- Stronger identity checks, for example comparing the face at the start with periodic snapshots, with careful attention to privacy and bias.
- Spectator mode and replays of the scoreboard after the contest.
- Native mobile applications for browsing and notifications.

- Machine-assisted similarity detection with explanations that help hosts judge a flag.
- Tools for colleges, such as batch invitations, class leaderboards and exports of results.

XVIII. CONCLUSION

This paper presented HackAgon, a platform that lets any user host a live coding contest while the platform guards the money and the fairness. The design combines a server-authoritative match engine, a sandboxed judge, an append-only ledger with integer arithmetic and idempotent webhooks, and a proctoring layer whose lock state lives on the server. A skill-only rule set, an age rule, transparent rules pages and a dispute path protect participants, and a Learning module with achievements gives players a reason to return.

The main engineering lesson of the planning stage is that the hard parts of such a system are not the pages but the invariants: the clock is the server's, the lock is the server's, and the money is whatever the ledger says. When these three facts hold, the rest of the system can stay simple. The team will build the product over twelve weeks in three milestones, collect the metrics defined in Section XV, and report the results in a follow-up paper. We hope the design is useful to other student teams who want to build contest platforms that are both fair and financially sound.

APPENDIX: REPRESENTATIVE INTERFACES

Table XV lists representative HTTP interfaces by route group. The list is a design-level summary of the contracts the team writes before connecting modules; exact paths and payloads are fixed in the shared contract file and may differ in the final code.

TABLE XV
REPRESENTATIVE INTERFACES BY ROUTE GROUP

Group	Operation	Notes
auth	Signup, verify codes, login, logout, forgot password	Rate limited; both channels verified
rooms	Create, list Arena, join by id or invite code, status	Server validates defaults and limits
play	Submit code, get verdict, scoreboard, remaining time	Server clock; one pending per question
proctor	Snapshot upload, violation log, lock state, heartbeat	Lock checked on every request
money	Create order, webhook, entry checkout, payout, refund	Signature check; idempotency key
content	Question search, question CRUD, library list	Filters: language, topic, difficulty
admin	Users, disputes, flagged cheating, payout failures	Role check on every route
ai	Hint after failed attempt	Daily limit per user

Each interface returns small JSON documents and uses a single error format, so that the front end can show a friendly hand-written message for each failure instead of a raw status code.

ACKNOWLEDGMENT

The authors thank their guide, Er. Ram Babu Buri, Department of CSE, and Arya College of Engineering and IT, Jaipur, for their guidance and support during this project, and the open-source communities whose tools make a project of this size possible.

REFERENCES

[1] Judge0, “Judge0 CE: open source online code execution system,” project documentation. [Online]. Available: https://judge0.com

[2] World Wide Web Consortium, “Page Visibility (Second Edition),” W3C Recommendation.

[3] World Wide Web Consortium, “Media Capture and Streams,” W3C specification (getUserMedia).

[4] S. Schleimer, D. S. Wilkerson and A. Aiken, “Winnowing: Local algorithms for document fingerprinting,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2003, pp. 76–85.

[5] Razorpay, “Razorpay developer documentation: Orders, Webhooks, Payouts and Fund Account Validation.” [Online]. Available: https://razorpay.com/docs

[6] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annual Technical Conf., FREENIX Track*, 1999, pp. 81–91.

[7] D. M’Raihi, M. Bellare, F. Hoornaert, D. Naccache and O. Ranen, “HOTP: An HMAC-based one-time password algorithm,” IETF RFC 4226, Dec. 2005.

[8] OWASP Foundation, “OWASP Top Ten Web Application Security Risks.” [Online]. Available: https://owasp.org/www-project-top-ten

[9] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. California, Irvine, 2000.

[10] HackAgon team, “HackAgon Product Requirements Document,” internal project document, 6 Oct. 2026.

[11] HackAgon team, “HackAgon Implementation Plan,” internal project document, 6 Oct. 2026.

[12] HackAgon team, “HackAgon Team Modules and 12-Week Plan,” internal project document, 8 Oct. 2026.

[13] Groq, Inc., “Groq API documentation.” [Online]. Available: https://console.groq.com/docs

[14] PingCAP, “TiDB Cloud documentation.” [Online]. Available: https://docs.pingcap.com/tidbcloud

[15] Brevo, “Brevo API and SMTP documentation (transactional email).” [Online]. Available: https://developers.brevo.com

[16] Render Services, Inc., “Render documentation: web services and deployment.” [Online]. Available: https://render.com/docs